

Spécialité : Transformation Digitale Industrielle
Module : Génie Logiciel
Compte Rendu du TP N° 02 :
Git + Node.js

Filière : TDI
Niveau : 2^{ème} Année

Effectuée par :

MOUGUERT Meryem

Encadré par :

Pr. ELBAGHAZAOUI Bahaa Eddine

A.U 2025/2026

Étape 1 : Initialisation du projet Node.js

Cette étape consiste à créer un projet Node.js versionné avec Git depuis zéro.

Commandes exécutées

```
$ mkdir mon-projet-node  
$ cd mon-projet-node  
$ git init  
$ npm init -y  
$ echo "# Mon Projet Node" > README.md  
$ echo "node_modules/" >> .gitignore
```

```
$ echo "console.log('Hello Node');" > index.js
$ git add README.md .gitignore index.js package.json
$ git commit -m "Initialisation du projet Node (README, .gitignore, index.js)"
```

Explication

git init	Initialise un dépôt Git local dans le dossier courant
npm init -y	Crée le fichier package.json avec les valeurs par défaut
.gitignore	Exclut node_modules/ du suivi Git (fichiers inutiles/lourds)
git add	Prépare les fichiers dans la zone de staging
git commit	Enregistre un instantané de l'état du projet avec un message

Contenu de index.js initial

```
console.log('Hello Node');
```

Étape 2 – Gestion des branches

On crée une branche dev pour développer une nouvelle fonctionnalité sans impacter la branche principale main.

Commandes exécutées

```
$ git branch dev
$ git checkout dev
$ git add index.js
$ git commit -m "Ajout de la fonction addition dans index.js"
```

Contenu de index.js (branche dev)

```
function addition(a, b) {
  return a + b;
}
if (require.main === module) {
  console.log("Résultat:", addition(5, 3));
}
module.exports = { addition };
```

Explication

git branch dev	Crée une nouvelle branche nommée dev
git checkout dev	Se positionne sur la branche dev
module.exports	Exporte la fonction pour pouvoir la réutiliser dans d'autres fichiers

Étape 3 – Fusion (Merge)

Une fois la fonctionnalité validée sur dev, on la fusionne dans main.

Commandes exécutées

```
$ git checkout main
$ git merge dev
```

Explication

Le merge intègre tous les commits de la branche dev dans main. Comme les deux branches n'ont pas modifié les mêmes lignes, la fusion se fait automatiquement sans conflit (fast-forward).

git checkout main	Retourne sur la branche principale
git merge dev	Intègre les commits de dev dans main

Étape 4 – Résolution de conflits

Un conflit Git survient quand deux branches ont modifié les mêmes lignes d'un fichier. Git ne peut pas choisir automatiquement quelle version garder.

Création du conflit

```
$ git checkout -b feature
Sur feature : ajout de la fonction soustraction dans index.js.
$ git add index.js
$ git commit -m "Ajout de la fonction soustraction"
$ git checkout main
Sur main : amélioration des commentaires de addition.
$ git add index.js
$ git commit -m "Doc: améliorer les commentaires de addition"
$ git merge feature
```

Message de conflit affiché

CONFLICT (content): Merge conflict in index.js
Automatic merge failed; fix conflicts and then commit the result.

Marqueurs de conflit dans index.js

```
<<<<<<< HEAD
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) { return a + b; // simple addition }
=====
function addition(a, b) { return a + b; }
function soustraction(a, b) { return a - b; }
>>>>>>> feature
```

Résolution : version combinée finale

```
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) { return a + b; // simple addition }
```

```
function soustraction(a, b) { return a - b; }
module.exports = { addition, soustraction };
$ git add index.js
$ git commit // valider la fusion
```

Explication

<<<<<< HEAD	Marque le début de la version de la branche courante (main)
=====	Sépare les deux versions en conflit
>>>>>> feature	Marque la fin de la version de la branche feature

Étape 5 – Rebase

Le rebase réécrit l'historique pour que les commits d'une branche apparaissent comme s'ils avaient été faits au-dessus d'une autre, rendant l'historique plus linéaire et lisible.

Commandes exécutées

```
$ git checkout -b bugfix
Sur bugfix : rendre addition robuste aux valeurs NaN.
$ git add index.js
$ git commit -m "Bugfix: addition robuste (gère les NaN)"
$ git checkout main
$ git rebase bugfix
```

Code après bugfix

```
function addition(a, b) {
  const x = Number(a), y = Number(b);
  if (Number.isNaN(x) || Number.isNaN(y)) return 0;
  return x + y;
}
```

Différence Merge vs Rebase

git merge	Crée un commit de fusion, préserve l'historique réel des branches
git rebase	Rejoue les commits au-dessus d'une base, historique linéaire et propre

Étape 6 – Cherry-pick

Le cherry-pick permet d'appliquer un commit précis d'une branche sur une autre, sans fusionner toute la branche.

Commandes exécutées

```
$ git log --oneline
3f5c2d3 Bugfix: addition robuste (gère les NaN)
1a2b3c4 Ajout de la fonction soustraction
9d8e7f6 Doc: améliorer les commentaires de addition
```

```
$ git checkout feature
$ git cherry-pick 3f5c2d3
```

Explication

On applique uniquement le commit de bugfix (3f5c2d3) sur la branche feature, sans récupérer tous les autres commits de main. Utile pour corriger un bug urgent sur plusieurs branches.

Étape 7 – Collaboration avec GitHub

On pousse le projet local vers un dépôt distant GitHub pour le partager et collaborer.

Commandes exécutées

```
$ git remote add origin https://github.com/VotreUser/mon-projet-node.git
$ git branch -M main
$ git push -u origin main
$ git config --global user.name "VotreNom"
$ git config --global user.email "VotreEmail@example.com"
```

Commandes de collaboration

```
$ git pull origin main # récupérer les modifications distantes
$ git push origin main # envoyer ses modifications
```

Bonnes pratiques

- Utiliser des branches par fonctionnalité : feature/nom-fonctionnalite
- Ouvrir des Pull Requests avant de merger dans main
- Protéger la branche main pour éviter les push directs
- Toujours faire git pull avant de commencer à travailler

Récapitulatif des commandes Git

git init	Initialise un dépôt Git local
git add .	Ajoute tous les fichiers au staging
git commit -m	Enregistre un snapshot avec un message
git branch	Crée ou liste les branches
git checkout	Change de branche
git merge	Fusionne une branche dans la courante
git rebase	Rejoue les commits sur une nouvelle base
git cherry-pick	Applique un commit précis sur la branche courante
git remote add	Lie le dépôt local à un dépôt distant
git push	Envoie les commits vers le dépôt distant
git pull	Récupère et intègre les changements distants
git log --oneline	Affiche l'historique compact des commits